

TECHNICAL RESEARCH & IMPLEMENTATION PLAN

ZenEngine

UI Integration

A cohesive editor workspace, not a debug overlay

DOCUMENT	REVISION	RESEARCH DATE
ZE-UI-001	0.1 • Proposed	30 September 2026

Executive decision

Adopt **Dear ImGui with docking** as the UI foundation. Build **ZenEditor** as a first-class editor application on top: workspace layout, scene and asset views, commands, selection, undo/redo, persistence, and specialized rendering tools. Do not make a general-purpose UI framework a prerequisite for editor development.

The desired result is one integrated application window with a central scene viewport and persistent docked panels. Docking and additional operating-system windows are separate capabilities; enable the former and leave multi-viewports disabled. [\[2\]](#) [\[3\]](#)

RECOMMENDATION IN ONE SENTENCE

Reuse UI mechanics; own the editing experience. Keep ZenEngine responsible for rendering and GPU scheduling, while a narrow adapter connects the editor UI to the existing engine.

What the first usable editor must demonstrate

Launch into a complete layout, open a scene, select an object, edit its transform, undo and redo the change, save, close, and reload. The viewport must resize with its panel, input must not leak between tools and camera controls, and the workflow must not introduce Vulkan validation errors.

Scope and confidence

This is an architecture proposal and implementation plan, not a report of completed integration. It uses the discussed C++/Vulkan, RHI, RenderGraph and resource-management context. No ZenEngine source checkout was inspected for this document; proposed class names, build targets and interfaces must be mapped to the current codebase in Phase 0. Public references support framework facts; design choices and acceptance criteria are recommendations.

Reading guide

- Requirements and technology decision • Sections 1–3
- Workspace, architecture and editing behavior • Sections 4–9
- Delivery, testing and risk controls • Sections 10–12
- Bootstrap example and primary sources • Sections 13–14

DEFINITION OF THE PRODUCT

01 Project context and requirements

Context carried into this plan

ZenEngine is a C++ rendering-engine project with a Vulkan backend, an RHI, a RenderGraph, and existing resource/upload infrastructure. The editor should exercise those systems rather than introduce a second rendering architecture. The active window/input implementation, exact frame-scheduling interfaces and existing scene serialization need repository verification.

Earlier project discussions included GLFW and consideration of SDL3. This plan does not treat either as a confirmed current choice. Retain the active platform layer for the first integration; a windowing migration is a separate decision.

Mandatory requirements

ID	REQUIREMENT	ACCEPTANCE INTENT
R1	Integrated workspace	One main OS window; main panels docked on first launch, with no persistent floating tool panels.
R2	Embedded scene viewport	Render the scene to an engine-owned image displayed inside the Scene View panel, not behind an overlay.
R3	Complete editing loop	Shared selection, transform editing, one undo step per gesture, dirty-state handling, and save/reload.
R4	Renderer ownership	ZenEngine retains device, swapchain, queue coordination and presentation ownership.
R5	Deterministic interaction	Text, modal dialogs, gizmos, camera navigation and shortcuts have explicit input priority.
R6	Runtime separation	A runtime/demo build can compile and run without the editor UI dependency.
R7	Recoverable layout	Persist user layout; reopen closed panels; reset invalid or outdated settings to a known default.

Initial scope

Provide Scene View, Hierarchy, Inspector, Assets and Console panels, a menu/toolbar, and a status area. The asset panel may initially browse the existing asset catalog; it does not require a new import pipeline. Establish one supported development configuration first, then validate any other platform already maintained by ZenEngine.

Explicit non-goals

Do not include detached OS tool windows, an embedded web application, a plugin marketplace, a new ECS, a full reflection framework, or a custom text/layout engine in the initial milestone. Runtime HUDs and player-facing menus remain a separate UI decision. Play mode, advanced material/node editors and multi-document workflows are later scope, not prerequisites for a usable scene editor.

ALTERNATIVES AND TRADE-OFFS

02 Framework research and comparison

The comparison below is an engineering assessment for ZenEngine, not a measured performance ranking. No prototype benchmark was run. A professional editor can be built with immediate-mode or retained-mode UI; the framework does not replace application design.

OPTION	VERIFIED FOUNDATION	ZENENGINE ASSESSMENT
Dear ImGui + docking	C++ tooling library; application-rendered draw data; docking branch and platform/renderer backends. [1] [2] [4]	Recommended. Fits an engine-hosted editor while leaving rendering ownership with ZenEngine.
Qt Widgets	QMainWindow provides a central area, menus, toolbars and dock widgets; Qt exposes accessibility infrastructure. [9] [11]	Credible alternative when desktop application behavior and assistive-technology support drive requirements.
Custom framework	An implementation project rather than a ready-made capability set. All required behavior must be built and tested.	Defer. It would combine editor development with a second, substantial platform/UI project.

Why Dear ImGui is the default

Dear ImGui is explicitly aimed at programmer tools and content-creation interfaces, and its core emits draw data rather than owning the graphics system. [\[1\]](#) For ZenEngine, that supports a small initial integration and direct access to rendering diagnostics. The application still needs a consistent visual language, command model, asset workflows and workspace rules.

When Qt becomes the better fit

Qt should be reconsidered before substantial UI investment when screen-reader integration or rich desktop interaction is mandatory. Dear ImGui documents limitations in accessibility and full internationalized text layout; that is a product constraint, not a styling issue. [\[1\]](#) [\[11\]](#)

A Qt-based editor need not discard ZenEngine, but the integration must explicitly resolve native window, event-loop and Vulkan ownership. QVulkanWindow manages device/queue/swapchain-related resources and documents limitations when embedded in QWidget interfaces. A custom QWindow integration is another route, not an automatic drop-in solution. [\[10\]](#)

Why not build the toolkit now

The work would extend far beyond rectangles and buttons: editable text, focus, keyboard navigation, clipping, popups, drag-and-drop, docking, DPI behavior, clipboard and platform input. Those responsibilities would compete with scene editing and renderer development. Build a custom toolkit only after an identified requirement fails a focused prototype, or when UI technology itself becomes a deliberate project goal.

ARCHITECTURE DECISION RECORD

03 Technology decision and dependency policy

ADR-UI-001 • PROPOSED

Use a pinned Dear ImGui docking revision, an existing compatible platform backend, and initially the official Vulkan renderer backend behind a ZenEngine-owned adapter. Keep a custom ZenRHI renderer backend optional, not inevitable.

REUSE	OWN IN ZENEDITOR	DEFER
Widgets, text controls, focus/navigation, clipping and docking mechanics.	Workspace policy, commands, selection, documents, inspectors, asset references and scene interaction.	A general-purpose UI framework; detached OS windows; a universal wrapper around every toolkit call.
A backend compatible with the engine's active windowing library. [4]	The boundary to native handles, queue scheduling, resource tracking and runtime build separation.	A window/input migration performed only to integrate the editor.

Pin a coherent dependency set

Record an exact tag or commit for the docking source, with matching core, headers and backend files. Keep an engine-owned configuration header and a small upgrade checklist. Do not build from a moving branch in reproducible builds, and do not mix a package-manager core with backend files copied from another revision. Retain third-party notices and record the selected extensions and licenses.

Compatibility findings from this research

The inspected public header identified itself as a work-in-progress version. It is evidence that development continues, not a recommendation to adopt that snapshot. Select the integration revision during Phase 0 and record the successful build/test result. [7]

The Vulkan backend changelog records an **April 22, 2026** change from combined image/sampler descriptors to separate sampled-image and sampler descriptors, including a changed texture-registration signature. Older tutorials can therefore specify the wrong descriptor types or arguments for a newer checkout. [6]

Current backend guidance also covers texture create/update/destroy requests used by dynamic fonts. A future custom renderer must implement the texture contract of its pinned revision, not only the traditional one-time font upload. [4]

Guardrails

Keep any DockBuilder/internal docking usage in one layout implementation unit; it is not a stable public API. [2] Keep Vulkan types out of editor-model interfaces. Do not fork upstream to establish a theme. Replace the renderer backend only when measured integration friction or a concrete platform requirement justifies the additional maintenance.

A DOCKED EDITOR, NOT FLOATING CONTROLS

04 Workspace and interaction design

REGION	DEFAULT CONTENT	BEHAVIOR
Top	Menu and editing toolbar	File, Edit, View and Help commands; selection/transform tools; document state.
Left	Hierarchy	Search, tree selection, rename and object visibility. Shared selection with Scene View.
Center	Scene View	Scene image plus editor interaction and viewport-local tools. The main working area.
Right	Inspector	Selected-object properties, validation and consistent transaction-based editing.
Bottom	Assets / Console tabs	Asset browsing and filtered logs. Bounded history and asynchronous asset status.
Status area	Project and renderer status	Active scene, background-job state and concise rendering information.

Docking policy

Ship a locked, fully docked default. Main panels may resize and switch tabs, but normal use must not create a persistent floating panel. Disabling multi-viewports only removes additional OS windows; it does not by itself enforce this workspace policy. [2] [3] Validate the selected revision's locking controls before declaring R1 complete.

For the first milestone, provide named layout presets rather than unrestricted drag-to-undock customization. Reopened panels return to their assigned region. Treat transient menus and modal confirmations as interactions, not as the main workspace. A fixed split-region layout within the same toolkit is an acceptable fallback if docking restrictions cannot reliably satisfy R1.

Initialization and persistence

Submit the dockspace before its child panels and maintain its lifetime when hidden. [2] Load user settings when compatible; otherwise create the default once. Store stable panel identifiers and a workspace schema version. Never rebuild the entire layout every frame, and do not silently replace a valid user layout at each startup.

Keep layout/settings files separate from scenes and per-user by default. Provide View > Reset Layout and View > Panels. Reject or repair settings that restore main tools as floating or inaccessible windows. Save failures must not prevent the application from launching with a usable default.

Visual language

Define shared typography, spacing, row-height, icon, border and selection tokens. Use aligned property labels, visible keyboard focus, meaningful empty states and consistent disabled/read-only states. Start with approximate left/center/right proportions of 18/58/24, then tune against real workflows. These are design starting points, not fixed pixel requirements; the viewport should keep the remaining usable space.

PROPOSED STRUCTURE – NOT EXISTING APIS

05 Module boundaries and ownership

MODULE	RESPONSIBILITY	DEPENDENCY RULE
EditorApplication	Lifecycle, active project, frame orchestration and shutdown.	Composes editor services; runtime does not depend on it.
EditorModel	Selection, documents, command history, dirty state and validation.	No Dear ImGui or Vulkan types.
EditorUI	Workspace, panels, property controls and tool presentation.	Uses Dear ImGui and model-facing services.
EditorRenderBridge	Viewport requests, UI texture registry, graph reads and draw submission.	The only editor layer allowed to translate to native backend handles.
Platform adapter	Window events, cursors, clipboard, focus and input-method integration.	Reuses the platform backend matching the active engine platform layer.
Engine / RenderCore / RHI	Scenes, rendering, resources, uploads and GPU scheduling.	Expose capabilities and data; do not include editor UI in runtime interfaces.

Build organization

Introduce an optional editor build target, for example ZenEditor, controlled by a proposed ZEN_BUILD_EDITOR option. Keep Dear ImGui linkage scoped to editor/integration targets. Existing non-editor demos must still build with the option disabled. Module names here describe responsibilities; they do not require renaming established ZenEngine directories.

Own meaningful abstractions

Create reusable editor controls such as TransformEditor, AssetReferenceField and ComponentInspector. Do not first create toolkit-neutral copies of Button, Checkbox and every other primitive. The useful independence is in editing operations and data, not in hiding all widget calls behind another incomplete UI API.

Threading contract

Start with one owner thread for ImGui state and UI submission. Workers publish completed asset/job results through an owned queue or snapshot; they do not mutate widgets or read mutable selection state. Apply editor commands at a defined boundary before building the scene render snapshot.

When rendering runs on another thread, either consume UI draw data before the next UI frame can modify it, or build a frame-owned packet with copied buffers, stable texture references and explicit callback lifetime rules. Do not queue raw pointers into transient UI storage. Keep GPU queue access serialized through the engine's submission owner.

Lifecycle

Stop new editor work, join/cancel pending jobs, wait for relevant GPU work, retire UI bindings and backend resources, destroy the UI context, then release device/platform services and custom allocators. Preserve dependencies during partial initialization failure as well as normal shutdown.

RENDER THE SCENE INSIDE THE WORKSPACE

06 Viewport and RenderGraph integration

RENDERING CONTRACT

Scene passes → engine-owned offscreen output → Scene View image → editor composition → main swapchain. The scene is content of a panel, not the background behind floating controls.

One-frame data flow

Begin the UI frame and determine the Scene View image rectangle. Acquire a valid per-frame viewport target before emitting the image widget. Collect interaction, then freeze the scene/camera state used for rendering. Finalize UI draw data, declare its sampled resources to the graph, build scene passes followed by editor composition, and let the existing scheduler submit and present.

Initially use persistent, per-frame viewport targets that are imported into the graph. This avoids needing a native texture descriptor before a transient graph allocation exists. Later, a logical UI texture handle can be resolved after graph compilation, provided the bridge owns that deferred mapping. Dear ImGui image APIs accept backend-interpreted texture references; they do not supply the engine's resource model. [8]

Make dependencies visible

Require every engine image displayed by the UI to register its resource handle, view/subresource and intended read usage with EditorRenderBridge. Before graph compilation, deduplicate these reads and declare them on the UI composition pass. Do not assume shader reflection of the engine's usual descriptor sets can see textures hidden in external UI draw commands.

For a color-attachment producer and UI fragment sampling on the same queue, the dependency connects color-attachment writes to fragment-shader reads with the appropriate image layout transition. Other producers, such as compute tone mapping, require their own actual stage/access scopes. Khronos provides matching synchronization examples. [12]

Extent and picking

Calculate render extent from the actual displayed image rectangle, framebuffer scale and an explicit render-resolution scale. Clamp to supported limits and skip scene rendering for a zero-sized or hidden viewport. Keep the main swapchain extent tied to the OS window; resizing an inspector is not itself a swapchain resize.

Map pointer coordinates through the image rectangle and any letterboxing before ray construction or ID-buffer lookup. Test at fractional display scale. Prefer an existing scene ray query for the first picker; asynchronous GPU picking must handle delayed results and stale entity IDs without blocking every click.

Color and sampling policy

Start with an SDR editor. Resolve multisampled scene output before regular texture display. Specify tone mapping, texture decoding, UI vertex-color treatment and final output encoding as one policy; verify that it avoids double gamma conversion. Test the chosen backend/shader path with reference patches rather than assuming the stock UI shader matches ZenEngine's color pipeline.

KEEP INTEGRATION NARROW AND EXPLICIT

07 Vulkan bridge and resource lifetime

Borrow engine infrastructure; do not duplicate it

Initialize the renderer adapter using the engine's device and a coordinated queue, and record UI commands into an engine-approved command buffer/rendering scope. Do not copy example helpers that create another main window or swapchain. The upstream Vulkan backend distinguishes integration APIs from example/window helper APIs. [6]

Match attachment format, sample count and render-pass or dynamic-rendering configuration to the actual UI target. Scope descriptor allocation and UI buffers to the bridge. After calling an external backend, invalidate or restore any RHI state cache that otherwise assumes pipeline, descriptors, viewport or scissor state remained unchanged.

A critical exception: backend-managed uploads

The inspected Vulkan backend's texture-update path includes queue submission and a queue-idle wait. Consequently, the stock backend is not automatically a pure RenderGraph recording callback. [6] Audit the selected revision and place the entire relevant backend operation under engine queue coordination; do not allow concurrent uncoordinated access from UI and render threads.

For the first integration, document backend-owned font/texture uploads as an explicit external operation. Profile cold and warm text paths. A custom backend is justified if those uploads must move into ZenEngine's staging/scheduling system or measured stalls cannot be addressed cleanly. Do not promise a stall-free renderer before this check.

Texture registry and retirement

Use a registry entry that maps a stable UI handle plus generation to an image view, backend binding, sampled subresource and ownership record. Cache bindings; do not allocate descriptors for every image widget every frame. A frame packet must retain the exact resource generation it references, not look up a mutable "current viewport texture" later.

On resize, allocate the replacement, publish a new binding, and retire the old image/view/binding only after all referencing submissions complete. Use the engine's completion tokens and deferred-release mechanism. Keep texture destruction distinct from removing a CPU-side registry entry. Routine resizing should not require a device-wide idle wait.

Frame and backend conformance checks

Ensure UI vertex/index buffers and bindings cannot be reused while the GPU still reads them. Check the backend's buffer ring against the engine's maximum outstanding UI submissions; swapchain image count and frame-slot identity are not interchangeable. For a custom backend, test clipping with display offsets/scales, index/vertex offsets, reset-state callbacks and the pinned texture-update contract. [4]

Begin with one composition call per main-window frame. Add multiple views or parallel UI recording only after the single-window lifetime and submission model is demonstrably correct.

ONE OWNER PER INTERACTION

08 Input routing and focus

Separate event ingestion from application action

Feed input events to Dear ImGui regardless of capture flags. The official FAQ uses capture state to determine whether the underlying application should also act on input, and warns about matching key-down and key-up delivery. [5] Keep this distinction in the platform adapter rather than scattering checks through camera code.

The embedded Scene View is itself part of the editor. It cannot simply use “capture is false” as its camera permission: the image lives inside UI. Route viewport actions through an explicit editor controller that considers the image item, focus, active tools and blocking popups. Do not bypass modal blocking with a global hover test.

PRIORITY	OWNER	POLICY
1	Modal or popup interaction	Block scene picking, camera motion and conflicting commands.
2	Active text/numeric edit	Reserve text and editing shortcuts; do not delete entities while editing a name.
3	Active gizmo or drag	Retain ownership until release, cancellation or application focus loss.
4	Scene viewport controller	Begin navigation or selection only inside an eligible image region; keep gesture ownership after leaving it.
5	Editor commands / gameplay	Dispatch permitted contextual commands. Gameplay remains inactive until an explicit future play/input mode.

Interaction state

Track pointer owner, active tool, focused document, active edit transaction and keyboard context. On focus loss, clear pressed-action state and release cursor capture. Escape cancels the active interaction before it triggers a broader command. Preserve the owner of a press when delivering its release, even if focus or hover has changed.

Implement shortcuts as command bindings, with platform-appropriate primary modifiers. A menu item, toolbar button and shortcut must invoke the same operation and share the same enabled state. During text editing, route Undo to the text field rather than silently undoing a scene operation.

Concrete behavior to verify

Scrolling the hierarchy must not zoom the camera. Right-drag camera navigation must finish correctly outside the image. Clicking a gizmo must not also select an object behind it. Opening a modal must prevent deletion and movement. Closing or unloading the selected object must cancel or safely resolve any pending edit.

Text and input methods

Use the established platform backend for clipboard, focus and input-method plumbing. Verify Unicode labels and any required input method on the supported platform. Do not equate basic Unicode/IME behavior with full complex-script layout or screen-reader support; those remain framework-selection gates. [1]

MAKE TOOLS OPERATE ON A COHERENT DOCUMENT

09 Editing model and persistence

Selection and object identity

Share a SelectionModel between hierarchy, viewport and inspector. Use stable entity identifiers with lifetime validation, not unchecked pointers retained by panels. Scope widget identity by object and property identifiers; names and collection indices can change during editing. Dear ImGui documents explicit ID scoping for avoiding widget identity collisions. [5]

Commands and transactions

Keep commands independent of widgets. Begin a transaction when a meaningful editing gesture starts, preview changes through a controlled document API, commit one before/after command when it finishes, and restore the original value on cancellation. A long transform drag must not create an undo entry for every frame.

STATE	DOCUMENT BEHAVIOR
Begin	Validate the target and capture the original value and document identity.
Preview	Apply valid temporary values; update the render snapshot without appending history entries.
Commit	Append one command only when the value changed; invalidate the appropriate derived data.
Cancel	Restore the original value; leave no committed history entry; release interaction ownership.

Define an explicit completion/cancellation rule for typed values, focus loss, selection changes and object deletion. Before saving, resolve any active preview transaction. Disable or reject commands whose target is no longer valid. Unit-test the command history without creating a UI or graphics device.

Inspectors and gizmos

Start with explicit inspectors for the existing scene components needed by the first workflow. Extract shared property metadata only when repetition demonstrates a need; do not block the editor on a general reflection system. ImGuizmo is a third-party candidate for transform manipulation, but matrix conventions, viewport bounds and undo integration remain ZenEditor work. [13]

Persistence rules

Separate scene content from user preferences and panel layout. Associate each editable document with its path, schema version, command history and saved revision. Dirty state must remain correct after undo/redo, branching history and failed saves; a failed write never marks the document clean.

Use a temporary-file-and-replace save strategy where supported, surface write errors, and preserve the last valid scene. Prompt before closing an unsaved document. Reuse existing serialization rather than inventing a new scene format just for the editor. Layout corruption should reset the workspace without affecting scene content.

Later play mode

Keep play mode outside the first milestone. When added, define whether runtime state uses a separate world or snapshot, how input ownership changes, and whether edits are intentionally applied back. Starting and stopping a simulation must not accidentally overwrite the authoring scene.

DELIVER BY EXIT CRITERIA, NOT CALENDAR PROMISES

10 Phased implementation plan

Implement phases in dependency order. The outputs below are planned deliverables, not existing features. The first usable editor is the exit from P3; later tools must not delay that complete editing workflow.

PHASE	IMPLEMENTATION OUTPUT	EXIT EVIDENCE
P0 Audit and spike	Map platform, frame slots, queue ownership and resource retirement. Pin the dependency set. Add an editor-only target and a minimal textured UI pass.	Clean editor/non-editor builds. UI renders in the existing main window. Record exact revisions and the backend upload/queue contract.
P1 Workspace shell	Create menus, stable panel IDs, default layout, closed-panel recovery, settings persistence and strict docked behavior.	Fresh launch, restart and Reset Layout produce a usable workspace; no main panel remains floating.
P2 Viewport integration	Add persistent frame-safe viewport targets, graph-declared UI reads, sizing, camera routing and a basic picker.	Resize, minimize, restore and high-DPI checks pass. Scene display is correct; UI input does not leak into camera control.
P3 Editing MVP	Connect shared selection, transform inspector, command transactions, undo/redo, dirty state and scene save/reload.	A recorded end-to-end workflow selects, edits, cancels, undoes, redoes, saves and reloads the same object correctly.
P4 Useful engine tools	Integrate the existing asset catalog, thumbnails, material properties and a first render-target/graph diagnostic view.	At least one real ZenEngine development task is completed through the editor without bespoke debug controls.
P5 Hardening	Run stress/performance tests, improve error recovery, verify supported platforms and evaluate backend replacement only against concrete requirements.	Validation, lifetime, recovery and performance evidence is attached; deferred limitations are explicit.

Small, reviewable changes

Keep platform/UI bootstrap, workspace layout, viewport resource handling and editing transactions in separate changes. Include a repeatable test scene and a short runbook with each integration milestone. Preserve working renderer demos so regressions can be localized to editor integration or shared engine code.

Scope-control checkpoint

At P3, assess the editor by completing an actual scene-editing task, not by counting panels or changing theme colors. Defer asset import redesign, node editors and plugin architecture unless the initial workflow proves they are necessary. Treat backend replacement as its own decision with a measurable benefit and regression-test burden.

PROPOSED TESTS — NO RESULTS CLAIMED

11 Validation and acceptance tests

TEST	SCENARIO	PASS CONDITION
T1 · Build boundary	Build with editor enabled and disabled.	Runtime targets have no accidental editor UI dependency; both configurations launch.
T2 · Layout policy	Fresh settings, restart, close/reopen panels, corrupted settings and Reset Layout.	A usable docked workspace is restored; no persistent floating or unreachable main panel.
T3 · Viewport extent	Repeated splitter/window resizing; minimize and restore.	No invalid zero-sized target, stale image or lifetime error; camera aspect follows the image region.
T4 · DPI and picking	100%, 150% and 200% scaling where supported; letterboxed image.	Pointer-to-scene mapping, clipping and gizmo alignment remain correct.
T5 · Input ownership	Rename, numeric edit, popup, camera drag, focus loss and gizmo drag.	No duplicate action, stuck key/cursor, accidental object deletion or blocked escape path.
T6 · Transactions	Long drag, typed edit, cancel, undo/redo, target deletion.	One history entry per committed edit; cancellation restores state; invalid targets are handled.
T7 · Save integrity	Edit/save/reload; failed write; close dirty document.	Saved data round-trips; failed saves preserve dirty state and previous valid content.
T8 · GPU safety	Validation enabled; multiple frames in flight; resize/unload with delayed completion.	No new UI-attributable validation errors, stale descriptors or premature reuse/destruction.
T9 · Bounded resources	At least 200 resize cycles and 100 panel open/close cycles after warm-up.	Live resource/descriptor counts return to a bounded baseline after GPU work retires.
T10 · Performance	Cold/warm text, scrolling, thumbnail loading and large hierarchy/log fixtures.	Record CPU/GPU percentiles, waits and memory. Meet the budget established on the named reference machine.

Measurement protocol

Capture an editor-disabled renderer baseline and the same scene in the editor at a matched scene-render resolution. Report UI CPU build time, UI GPU composition time, draw/vertex counts, descriptor count, allocation count and p50/p95/p99 frame time. Separate shader/font/asset warm-up from steady-state interaction.

Use provisional stress fixtures such as 1,000 scene objects and 10,000 asset entries to expose unbounded traversal or synchronous thumbnail work. These are test inputs, not claims of supported scale. Set numeric budgets in P0 after choosing the reference hardware and build configuration; do not invent a universal millisecond threshold.

Known baseline validation messages must be documented separately, not hidden. No new validation errors is necessary evidence, but not proof of complete correctness; retain the interaction, persistence and lifetime tests.

KEEP THE RECOMMENDATION REVERSIBLE

12 Risks and decision gates

RISK	CONTROL	REVISIT TRIGGER
UI work expands into a toolkit project	Use library primitives; implement domain-specific controls and the editing loop first.	A mandatory interaction fails a focused prototype.
Backend conflicts with GPU scheduling	Audit uploads, waits and queue access; serialize through the engine submission owner.	The official backend cannot satisfy the required scheduling contract without intrusive changes.
Hidden UI reads break graph correctness	Register every displayed engine image and declare composition-pass reads.	A new widget or callback bypasses resource tracking.
Layout is technically dockable but still floats	Test strict locking, recovery and presets as product behavior.	Pinned docking behavior cannot meet R1; use fixed split regions or reconsider the shell.
Dependency updates break integration	Pin core and backends together; isolate internal API usage; rerun the regression suite.	An upgrade changes texture, docking or backend contracts.
Desktop accessibility/text requirements change	Resolve mandatory assistive-technology and text-layout needs before P1. [1] [11]	Requirements exceed verified Dear ImGui support; prototype the Qt route.
Resource growth or interaction stutter	Bound logs/thumbnail caches; load asynchronously; profile cold and warm paths.	Stress runs reveal unbounded growth, repeated allocation or frame-time spikes.

Decisions to resolve during P0

Confirm the active window/input backend; the owner of Vulkan queue submission; the safe extension point for external draw recording; the frame-slot/completion-token model; and the existing scene serialization API. Choose the first supported platform and the exact pinned UI dependency revision. These are source-code verification tasks, not reasons to redesign the engine in advance.

When to implement a ZenRHI UI backend

Proceed only for a clear requirement: another maintained graphics backend, strict graph-managed uploads, integration with engine memory accounting, or a demonstrated scheduling/performance issue. Keep EditorModel and panels unchanged. Replace only the renderer adapter, retain the platform backend where possible, and rerun T3–T10.

Decision summary

The recommended investment is a small, well-tested UI bridge plus a substantial editor application. Its success is a reliable authoring workflow inside a coherent workspace. Dear ImGui is the foundation, not the product; ZenEditor's layout policy, commands and resource/input discipline make it an engine editor.

IMPLEMENTATION APPENDIX

13 Bootstrap and first changes

The snippet below shows only toolkit configuration and dockspace submission. It is not a complete engine integration, default-layout builder or locking implementation. Use headers and backends from the same pinned docking revision. The named flags and dockspace helper are documented upstream. [\[2\]](#) [\[3\]](#) [\[7\]](#)

```
#include "imgui.h"

// Call once, after creating the ImGui context.
void ConfigureEditorUi()
{
    IM_ASSERT(ImGui::GetCurrentContext() != nullptr);
    ImGuiIO& io = ImGui::GetIO();
    io.ConfigFlags |= ImGuiConfigFlags_DockingEnable;
    io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
    io.ConfigFlags &= ~ImGuiConfigFlags_ViewportsEnable;
}

// Call after NewFrame(), before submitting editor panels.
void SubmitEditorDockspace()
{
    ImGui::DockSpaceOverViewport();
}
```

This configuration does **not** prevent floating panels inside the main window. Add the validated layout/locking policy from Section 4. Integrate toolbars and status areas into a deliberate host layout; the helper alone does not define the finished editor shell.

First implementation changes

CHANGE	CONTENT AND REVIEW EVIDENCE
1 · Dependency and build	Pinned source/configuration, compatible platform backend, editor-only target and non-editor build proof.
2 · Native bridge	Existing-device initialization, coordinated queue usage, correct rendering scope and clean shutdown.
3 · Workspace	Default docked layout, stable panel IDs, settings version, panel recovery and reset behavior.
4 · Viewport	Frame-safe offscreen image, texture registry, explicit graph reads, correct sizing and input routing.
5 · Editing workflow	Shared selection, transform transaction, undo/redo, save/reload and repeatable test scene.

Implementation note

Avoid copying initialization structs, font-upload calls or texture-registration signatures from unrelated tutorials. Read the exact backend files you pinned. Keep native integration and any internal docking API usage localized so future upgrades do not require editing every panel.

14 Primary references

Numbered citations identify upstream capabilities and API behavior. Architecture, milestones, test workloads and module names are proposed for ZenEngine. Branch documentation can change; the implementation must retain an exact dependency revision.

[1] [Dear ImGui – project README](#)

Purpose, draw-data model, tooling focus, accessibility and internationalization limits.

[2] [Dear ImGui – Docking wiki](#)

Docking branch, dockspace ordering, layout persistence and internal DockBuilder status.

[3] [Dear ImGui – Multi-Viewports wiki](#)

Additional OS-window behavior and the separate enablement flag.

[4] [Dear ImGui – BACKENDS.md, docking branch](#)

Platform/renderer separation, reuse guidance, custom renderer and texture-update contracts.

[5] [Dear ImGui – FAQ.md, docking branch](#)

Input capture, event forwarding, key-release caveats and widget ID scoping.

[6] [Dear ImGui – Vulkan backend source and changelog](#)

April 2026 descriptor/signature changes, integration helpers and texture-upload queue behavior.

[7] [Dear ImGui – public header, docking branch](#)

Configuration/API declarations and the inspected development-version marker.

[8] [Dear ImGui – Image Loading and Displaying Examples](#)

Application textures and backend-interpreted texture identifiers.

[9] [Qt – QMainWindow class, Qt for Python 6 documentation](#)

Main-window layout, central content, toolbars, dock widgets and saved state. The plan proposes C++ Qt Widgets, not a Python dependency.

[10] [Qt – QVulkanWindow class](#)

Managed Vulkan resources, rendering lifecycle and QWidget embedding limitations.

[11] [Qt – QAccessible class](#)

Interfaces connecting applications to assistive technologies.

[12] [Khronos – Vulkan Synchronization Examples](#)

Attachment-write to sampled-read dependencies and image layout transitions.

[13] [ImGuizmo – project repository](#)

Dear ImGui-based 3D transform-manipulation extension to evaluate.